Escurel: A Typed Knowledge Base for Agents

Skills and Instances, Events and Callbacks, and One Referent Space over Five Storage Substrates

Joachim Rosskopf
DataZoo GmbH
Germany
jr@data-zoo.de

Abstract

We present `escurel`, a knowledge base built for agents rather than for people, and report what ten weeks and 101,500 lines of Rust taught us about it. The system rests on two dualities. A skill page such as `customer` is both the schema its instances must satisfy and the prose an agent reads to learn what a customer is for, and an instance such as `customer::acme-corp` is both a row of that type and a markdown file somebody edits in a text editor. The second duality is temporal: events are immutable facts that land in an inbox, and the agent run that folds one into instance state takes its instructions from that event's own skill page. The first duality is what bounds context cost: because discovery happens at the level of types, an agent's cold-start cost tracks the number of skills, which people author and which therefore stops growing, rather than the number of instances, which machines generate. A corollary follows that we did not anticipate and that turned out to matter more than either duality alone. Since a skill declares its own backend, an instance's storage substrate is a property of its *type*. The same `resolve`, `expand` and `neighbours` calls reach native markdown, a read-only SQL view over Postgres, an extracted PDF, a live REST endpoint and an upstream MCP server; only similarity search stops at the three substrates that are materialised, which is the one place the abstraction is visible from the agent's side. We give the design, a register of eighteen safety and semantic guarantees in which every row names an integration test that actually runs, and an evaluation protocol covering context cost at five corpus scales, retrieval against flat retrieval over identical bytes, and the cost of each substrate. We also report what went wrong: the default embedder our own specification pins cannot be loaded by our own runtime, and lexical retrieval collapsed to 0.350^{p} nDCG on a duplicate-stress corpus where vector retrieval held at 0.933^{p}.

1 Introduction

Agent memory is a context problem, not a storage problem. Storing a million facts is not hard and has not been hard for decades. What is hard is getting the right handful of them into a bounded context window, over and over, cheaply enough that an agent can afford to look. Every retrieval a language model performs is paid for twice, in tokens and in turns, and both prices are set by how much the knowledge base makes the agent read before it can act. Flat vector memory — chunk the corpus, embed the chunks, retrieve by cosine similarity — answers exactly one question, "what text resembles this text," and charges a document body for every answer. It has no notion of what kind of thing it just returned.

Two dualities. We argue that two dualities, and one corollary of the first, are what let a knowledge base grow without the agent reading more to use it. The first is static: a *skill* is simultaneously a type (the frontmatter schema a class of pages must satisfy) and a prompt (prose that tells a model what this kind of thing is for), and an *instance* is simultaneously a row of that type and a markdown document a person can edit in any text editor. The second is dynamic: an *event* is an immutable fact that lands in an inbox, and a *callback* is an agent-harness run that projects it into instance state, taking its instructions from a skill page. Neither works alone. Skills without events give you a wiki that rots; events without skills give you an append-only log nobody can query.

The corollary: not everything is markdown, and it never will be. An organisation's knowledge does not live in one place. Some of it is prose, some is a Postgres table, some is a PDF nobody has read since it was signed, and some exists only behind a CRM's REST API. Because a skill declares its own `backend:`, the substrate an instance's data lives on is a property of the instance's *type* rather than of the referent space. One skill's instances are markdown; another's are rows of a read-only SQL view; another's are extracted document chunks; another's are fetched live from a REST endpoint on every read, and accept writes back upstream. The agent resolves, expands and traverses all of them with the same calls, and nothing in its prompt branches on where the bytes came from. There is one exception, and we would rather name it here than bury it: the two live substrates are never indexed, so similarity search does not reach them.

What we built, and what it cost. `escurel` is 23 Rust crates and roughly 101,500 lines, with 172 integration-test files, written over ten weeks. Three applications run on it and drove most of its surface: a knowledge explorer, a document and chat workspace, and a report renderer. It deploys as a pet rather than cattle, pinned to one host and stopped before it is replaced, because a DuckDB file has exactly one writer and we would rather make that visible in the deployment than paper over it in the code. Storage is one DuckDB file per tenant carrying relational metadata, an HNSW vector index, a BM25 index and a CRDT operation log, beside a `pages/` markdown directory that remains the canonical

Numbers marked x^{p} are prototype-era: they come from the 430-line Python predecessor [14] and its verification tree, not from the system described here. Numbers written **[? like this]** are specified in Section 6 but not yet measured.

source of truth. Every agent that writes to the corpus runs in a separate process, so the store has no background writer of its own. That constraint cost us a component and bought us the ability to reason about the store without reasoning about the loop.

Contributions.

- The two-duality model and its substrate corollary, with the mechanism by which typing, not disclosure discipline alone, is what bounds an agent's context cost (Sections 2–3).
- A design that realises it: typed wikilinks validated at index time, one embedded file carrying all five index concerns, five instance backends behind one referent space, and a reactive projection loop whose reducer generalises from one-hop cascades to fan-out workflows (Section 4).
- A guarantee register comparing `escurel` against flat vector memory in which every row names an integration test that actually runs (including the row where the test we had was asserting the wrong thing) (Section 5).
- An evaluation protocol, specified to the level of the harness invocation, covering context cost at five corpus scales, retrieval against flat retrieval over identical bytes, and a five-substrate cost profile (Section 6).
- Negative results, reported because each was the natural thing to believe (Section 6, Section 7).

What this draft can and cannot defend. The design, the guarantee register and the failures we report are complete. The measurements are not. Section 6 specifies each experiment down to its harness invocation, and every cell we have not run prints in red instead of being described in prose. On the evidence here we can defend an architecture and the discipline used to verify it. The claim that this bounds an agent's context cost at scale rests on Section 6's first experiment, which is specified and not yet run. We would rather say so than let the surrounding argument imply otherwise.

2 Background: What Flat Memory Costs

The cost model. An agent's cost on a task is the number of tokens it carries in context multiplied by the number of turns it takes. Both are functions of one design choice: how much the knowledge base makes it read before it can decide what to read next. We call the metadata an agent loads at cold start *Tier 1*, and the bodies it later chooses to pull *Tier 2*. The distinction is Anthropic's [2], applied to procedural knowledge; the question this paper asks is what it takes to apply it to episodic knowledge, which is where the volume actually is.

Flat vector memory has no Tier 1. Without types there is nothing to enumerate. There is no set of a hundred things an agent can hold in mind while it decides which of a million things to fetch, so every question becomes a similarity query over the whole corpus and every answer arrives as a body. Table 1 gives the arithmetic that follows, measured on the predecessor's corpora [14]. The numbers are prototype-era and the ratio is what matters: at ten thousand pages, reading metadata only costs three to five thousand tokens, and reading everything costs four hundred to six hundred thousand.

Table 1: What a cold start costs, by disclosure strategy. Prototype-era measurements over the predecessor's corpora, `tiktoken cl100k_base`. The working budget is a nominal 30k tokens.

strategy	10k pages	100k pages
every page, full content	400–600k^{p}	4–6M^{p}
every page, metadata only	3.2–4.8k^{p}	8–12k^{p}
every *skill*, metadata only	189^{p}	189^{p}

The third row is the one that motivates the paper. It does not grow, because what it enumerates is not the corpus.

Flat memory also has exactly one substrate. This gets less attention than the token arithmetic and causes at least as much trouble in practice. A flat vector store can reason only about text it has already ingested, so every table, every document and every record behind an API must first be copied in and chunked. The copy is stale the moment it lands, the pipeline that refreshes it is a component somebody has to run, and the parts of the business that change fastest are usually the parts that are most expensive to keep copied. Section 3 argues that this is not an ingestion problem to be solved with better pipelines but a typing problem, and that once types exist it mostly dissolves.

"Use a bigger window" is not an answer. Long-context models make the naive strategy possible at small scale and still not advisable: cost and latency grow with the window, and retrieval accuracy degrades for material in the middle of it [8]. More to the point, the strategy has no stable operating point. A corpus that fits today does not fit next quarter, and the failure is silent.

What a write costs. Flat memory's write path is "embed and append." Nothing validates the new material against anything, nothing supersedes what it replaces, and nothing links it to what it concerns. The corpus therefore accumulates contradictions at exactly the rate it accumulates content, and the retrieval layer, which ranks by similarity, is structurally unable to prefer the correct one of two similar answers.

And then it goes stale. The complement is just as bad. A typed knowledge base with no ingestion loop is a wiki: correct on the day it was written, quietly wrong six weeks later, and trusted for far longer than that. Any model of agent memory that stops at the data model has described half a system. That is why this paper has two dualities in it and not one.

3 Two Dualities and a Corollary

3.1 Skill ↔ Instance

The observation the system rests on is small and, once made, hard to unsee: an Agent-Skill page and a personal-wiki note

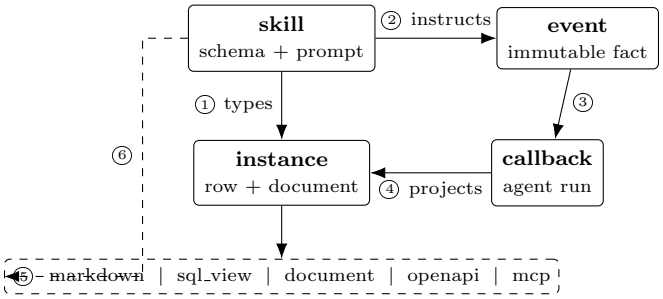

Figure 1: The model. A skill types its instances ① and instructs the callback that handles an event of that kind ②; an event fires a callback ③, which projects it into instance state ④. An instance's data may sit on any of five substrates ⑤, chosen by the *skill* ⑥ and never by the caller.

are the same artefact. Both are markdown with frontmatter, an identifier and a description. The only difference is that one is read as instructions and the other as data. So we let an instance page declare which skill it conforms to, and the skill becomes that instance's type.

The type is carried by the link, not inferred from it. References take the form `[[skill::id]]` and are validated when the page is indexed. Four checks run on every typed link: the skill page exists, the target page exists, the target's own `skill:` frontmatter field equals the link's skill segment, and any named anchor is present on the target. A link that fails one is reported, never silently dropped. The skill segment is then stored as a column on the link row, which makes "who cites any decision record" an indexed relational query rather than a graph walk.

Why this bounds context cost. Discovery enumerates skills. Skills are written by people. They are the vocabulary of a domain, not its contents, so their count is bounded by human attention rather than by ingestion rate. A tenant that ingests a million meeting notes still has one `meeting` skill. This is the whole mechanism, and it is worth being precise about what it does and does not claim: progressive disclosure bounds cost *only because* typing gives it something to disclose about. Disclosure discipline applied to an untyped corpus has no level of description to stop at.

Concretely: an agent asked what changed with a customer last quarter begins by listing skills, which returns on the order of a hundred identifier and description pairs and no bodies at all. It reads `meeting` and `decision-record` out of those descriptions, issues one `list_instances` call filtered by skill and date range, and expands the two or three pages it actually needs. The other several hundred thousand instances cost it nothing, because no step in that sequence has a size that depends on how many there are.

Links carry anchors and versions too. A reference splits on `|` (alias), `#` (anchor), `@` (version) and `::` (skill), in that order, giving six usable forms from the bare `[[skill]]` up to the full `[[skill::id#anchor@version|alias]]`. An anchor addresses a block within a page; the version pins a snapshot. Typing also closed a real class of ambiguity: in the untyped predecessor, `[[Anna Mueller]]` and `[[anna-mueller]]` resolved to two different nodes, because nothing said what a canonical form was.

3.2 The Type Declares Where the Value Lives

Skills carry a `backend:` block, and the skill is where it is carried: never the instance, never the caller. Five substrates exist today:

- `markdown` — the default. Writable at block granularity through a CRDT session, indexed, searchable.
- `sql_view` — a read-only DuckDB `VIEW` over an external relational source (Postgres, MySQL, SQLite, an ERP connector, or directories of JSON or Parquet). Materialised, so it is searchable.
- `document` — an uploaded PDF, DOCX, PPTX, XLSX or text file, extracted, chunked and embedded into one page with blocks.
- `openapi` — a live REST endpoint. Nothing is stored: the body is fetched on every `expand`, and writes are forwarded upstream.
- `mcp` — a live upstream MCP [1] server, read as a tool call or a resource, with the same write-back path.

One invariant makes this cheap. Every external instance keeps a markdown overlay page. The page *is* the instance as far as the referent space is concerned: identity, links, access control and history all reuse machinery that already existed. A `backend_ref` block binds the page to the external data. All the novelty is confined to where the body comes from. That is why adding a backend required no change to the dispatcher, no change to the wire format and no new tool for the reading agent: `resolve`, `expand`, `neighbours`, `list_*` and `search` route through the backend transparently.

The division that does show through is materialisation. `sql_view` and `document` are materialised and therefore indexed and searchable. The two live proxies are not: their data never enters DuckDB, so they report `search: "none"` and an agent cannot find them by similarity, only by link or by the overlay page's own metadata. We regard this as an honest limit rather than a defect (Section 7), but it is the one place where the substrate is visible from the agent's side, and we say so rather than claiming a transparency we do not have.

Time travel is expressed in the same syntax. A link may pin a version: `[[table::customers.churn@v14]]` resolves against snapshot 14 of an attached DuckLake [5] catalog, and `@2026-04-12` pins by date. For markdown instances the segment is accepted and ignored. One notation covers "the current state of a customer record in our CRM" and "the churn table as it stood the morning the decision was taken."

3.3 Event ↔ Callback

An event is an instance of an event-typed skill — `meeting`, `email`, `incident`, `commit` — carrying an `at:` timestamp and typed links to whatever it concerns. Events are immutable by convention: a correction is a new event that cites the old one, not an edit. The inbox is simply the suffix of the log that has not been projected yet.

The callback's instructions are a skill page. When an event arrives, a runner triggers an agent harness whose task description *is* the event's `label_skill` page and whose toolset is the ordinary MCP endpoint. There is no bespoke per-harness client and no prompt template maintained outside the corpus. The instructions for processing meetings live in the `meeting` skill, where a human can read, review and edit them, and where they are versioned with everything else.

State lives in the knowledge base, not in a script. This is the inversion that makes the loop recoverable. A workflow engine that holds progress in local variables loses it when the process dies. Here, progress is events and instances, which are durable, queryable and replayable, so a crashed run resumes by re-reading what it already wrote.

3.4 The Cross Product

The two dualities are not independent contributions that happen to share a repository. Each supplies something the other needs.

The skill-as-schema is what the callback's write is validated against; the skill-as-prompt is what drives the callback in the first place. The event is the instance's provenance; the callback is how the instance acquires a current value at all. And the corollary is what lets the loop reach past the corpus: because an instance may be a live REST record, a callback can fold an event into a system `escurel` does not own, writing back through `write_instance` rather than into a copy that will drift.

Drop the corollary and the other two still work, but only over material the system has already imported, which is where most retrieval-augmented architectures sit and why so much of their engineering turns out to be pipeline engineering.

4 Design and Implementation

4.1 One File, Five Index Concerns

Each tenant is a markdown directory beside a single DuckDB [12] file. That file carries relational metadata and the typed link graph; a table of blocks, one row per markdown block, whose vector column is indexed by HNSW through the `vss` extension and whose body column is indexed by BM25 through `fts`; the CRDT operation log; and an attach point for read-only lakehouse catalogs. A filtered vector search is therefore one SQL statement, because the filter columns are denormalised onto `blocks` beside the vector.

The predecessor split this across LanceDB for retrieval and DuckDB for relational state, bridged through Arrow, with CRDT operations in sidecar files. Lance won the original benchmark: 0.975^{p} nDCG, tying exact KNN, at roughly half the p50 latency and a sixth of the disk. We collapsed it anyway, accepting a measurable latency increase in exchange for deleting a storage engine, a cross-store consistency boundary, a sidecar atomicity argument and a three-way audit. The thresholds at which we would have reverted were declared in advance; Section 6 reports where we stand against them, which is not where we would like.

The `pages/` directory stays canonical. Everything in the database is derivable from it, which means `git diff` still works on a corpus, Obsidian still edits it, and recovery is a rebuild rather than a restore.

4.2 Typing at Index Time

The wikilink parser is a regular expression rather than a markdown AST walk. Markdown parsers split `[[` across nodes in ways that depend on the surrounding context, so the bracket that opens a reference and the one that closes it need not land in the same node. We evaluated the AST path and abandoned it before writing the indexer. The four validation checks run in the same pass, and their output is the same issue list that the `validate` tool returns on a dry run, so an authoring agent gets exactly the feedback the indexer would give it, before it writes.

4.3 One Referent Space, Five Substrates

The corollary of Section 3.2 is where a security reviewer should look first, because it is the part of the system that makes outbound network calls on behalf of tenant-authored content. Five invariants keep it safe, and each is a test rather than an intention.

Secrets never enter the corpus. A `sql_view` names a credential registered out of band by an administrator and realised as a DuckDB secret; the listing tool returns names only. Markdown never contains a password, so exporting a tenant never exports one.

There is no SSRF surface. A remote skill names an *administrator-registered endpoint*, not a URL. Tenant markdown cannot cause the server to fetch an arbitrary host, and a skill whose declared kind disagrees with the kind its endpoint was registered under fails closed rather than guessing.

A failed read never fabricates a body. If a live source times out or errors, `expand` returns the overlay page plus an explicit issue marker, and E-15 is that path under an unreachable host. A knowledge base that answers confidently from a source it could not reach is worse than one that is down.

Writes are value-bound. Remote write-back fills path and body templates from scalar payload fields; an exact leaf keeps its JSON type, and a placeholder that cannot be resolved aborts the call rather than sending a literal brace to an upstream system.

Schema drift is detected, not tolerated. A `sql_view` captures a fingerprint of its source schema at creation and revalidates it on attach, marking the binding degraded when the source has moved underneath it.

4.4 The Write Path

Live editing is a CRDT operation stream over Loro [9], persisted transactionally with the relational changes so that a commit is atomic across pages, links, blocks, indexes and operations together. A whole-page `update_page` fallback exists for clients that cannot hold a session. Writes take a per-tenant write lock; reads do not take it at all. Markdown is written with write-then-rename, and only after the database commit succeeds, so a crash leaves the two reconcilable in one direction, which is the direction the rebuild runs.

The non-markdown backends are read-only on this path by construction. Remote write-back is a separate, explicitly named tool, gated by the target instance's own access control. We considered making `update_page` transparently write through to a REST endpoint and rejected it: a page edit and an API call have different failure modes, and papering over that with a shared verb would have made both harder to reason about.

4.5 All Automation Outside the Gateway

The knowledge base schedules no work against tenant content. A separate runner process subscribes to an outbound webhook and also polls the inbox (the poller is the self-healing path for a webhook that was missed), and both converge on one bounded per-tenant queue. A runner-local durable ledger, deliberately not in the tenant store, holds one row per run and is the basis of every loop control: idempotency, deduplication, depth and budget caps, cycle detection.

One periodic task does run inside the gateway, and naming it is the point: a reader replica polls for newly published snapshots and hot-swaps its index. It only ever reads, and it exists because a read replica that never notices a new snapshot is not a replica.

We were tempted more than once to move the rest inside as well. Keeping it out means the store has no background writer of its own, so "what is in the knowledge base" is a question about the knowledge base and not about a scheduler's state.

4.6 One Loop at Two Settings

The cascade emitter is already a reducer over a run's event log, just a degenerate one: it emits at most one follow-on event per confirmed cross-skill write, with no join. A multi-phase workflow is the same reducer with three knobs turned up: an explicit plan read from a workflow skill, fan-out width greater than one, and a quorum barrier where a phase waits for k sibling completions.

Fan-out orchestration is currently being built as a second execution model beside the reactive one, and it does not need to be. Running a workflow is itself an inbox event; the only thing separating it from an externally captured one is who captured it.

4.7 Packs: Shipping Types, and Their Bindings

Skills are distributable. A pack is a versioned, read-only layer of pages that a tenant subscribes to; local pages shadow base pages of the same name, promotion back into the base layer passes a curator gate, and a pinned version never moves on its own. With the corollary in play, a pack ships more than a schema: a CRM pack carries skills whose instances resolve against *the subscriber's* registered endpoint. The type library and the integration arrive together, and the subscriber supplies only the credential.

4.8 What We Refuse

No raw SQL on the agent surface, no direct vector access, no cross-tenant operation, no auto-provisioning on first request, and no raw URL in tenant markdown. Each refusal removes a capability that would have been convenient and an attack or corruption path that would have been permanent.

5 Guarantees and Their Verification

A knowledge base is a contract. Tests passing is not the same as a contract being stated, and a contract nobody wrote down is one the next component will violate. Table 2 states what `escurel` promises, what a flat vector store promises for comparison, and for each row the test that verifies it. Every named test runs in the ordinary workspace test command; none is behind an ignore attribute, because a claim whose test does not run is not a claim.

On E-18, which is the row that changed shape. Our specification describes a tenant manager holding a bounded cache of per-tenant handles, each with its own pool and write lock. It is not implemented. The binary wires one shared indexer and one served tenant, so the deployed isolation boundary is the process, not a data structure inside it: one instance, one tenant. That suits a per-tenant deployment model well enough, but it is not what the specification says.

We learned this the way these things are usually learned. Authentication resolved a tenant from the token's claim and the data path used the served tenant, and for some weeks nobody compared the two. A validly signed token for tenant B, presented to tenant A's gateway, was accepted, had its quota debited against B, and then read and wrote A's corpus. What makes it worth recounting is that a test had encoded the bug as intended behaviour: a foreign token succeeding was being used to demonstrate that quota buckets were per-tenant. The gateway now rejects a mismatch with 403 for every role, including administrators, and E-18 is that test rewritten. A guarantee register is worth having mostly because it forces this comparison; its characteristic failure is becoming a place where aspirations get recorded as properties.

Remote backends are tested against real servers. The tests behind E-14–E-16 start actual HTTP services and an actual upstream MCP server in-process, with authentication, unauthorised responses and unreachable hosts among the cases. This follows a project rule that a test which mocks the boundary it exists to cover is not finished. It is more work per test and it is the reason we trust these five rows more than we trust our own architecture diagram.

Table 2: The guarantee register. Every row names an integration test that runs in `cargo test --workspace --all-targets`. A flat vector store over the same corpus provides none of these except a durable append (E-5): it has no typed reference to resolve, no schema to validate against, no session to converge, and no binding to degrade.

	promise	verified by
E-1	`resolve` of a valid typed link is deterministic and total	`resolve_typed_wikilink_to_known_page`
E-2	a link failing any of the four checks is reported, never dropped	`validate_instance_with_unknown_declared_skill_errors`
E-3	anchors and versions survive resolution unchanged	`resolve_typed_wikilink_with_anchor_and_version_keeps_segments`
E-4	concurrent CRDT sessions on one page converge	`concurrent_apply_ops_serialise_through_actor`
E-5	a mid-write crash rolls back every table together	`mid_write_panic_rolls_back_all_tables`
E-6	the index is reproducible from markdown alone	`node_loss_then_rebuild_from_markdown_reproduces_index`
E-7	a stale base version is rejected rather than silently merged	`version_advances_and_stale_base_version_conflicts`
E-8	one event yields exactly one terminal run, however delivered	`same_event_twice_yields_exactly_one_terminal_run`
E-9	a cascade terminates on depth or cycle, and is dead-lettered	`cascade_cycle_is_dead_lettered_at_depth_or_cycle`
E-10	a workflow phase completes only when its barrier quorum is met	`verify_barrier_runs_to_completion_via_echo`
E-11	every non-markdown backend refuses page-level writes	`update_page_on_sql_instance_rejected_backend_read_only`
E-12	a subscribed pack's base pages cannot be written or forged	`update_page_cannot_fabricate_a_base_layer_page`
E-13	credentials never appear in the markdown corpus	`secret_is_server_side_only_never_in_markdown_corpus`
E-14	a remote read fails closed on a kind mismatch	`remote_read_fails_closed_on_endpoint_protocol_mismatch`
E-15	an unreachable endpoint degrades the read, never fabricates it	`openapi_unreachable_endpoint_degrades_read_and_probe`
E-16	an unresolvable write placeholder aborts the upstream call	`mcp_resource_read_fails_closed_on_unresolved_placeholder`
E-17	source schema drift marks the binding degraded	`schema_drift_marks_binding_degraded`
E-18	a token minted for another tenant is refused, for every role	`foreign_tenant_rejected_and_served_tenant_quota_enforced`

6 Evaluation

6.1 Setup and Protocol

Retrieval runs use a 768-dimensional BERT-family encoder, `BAAI/bge-base-en-v1.5`, and a cross-encoder reranker from the same family. Two constraints pin that choice and are worth stating because one of them is a defect: the block vector column is fixed at 768 dimensions, and our Candle-based loader supports BERT architectures only. Datasets are in BEIR format [15], the corpus identifier is used verbatim as the page identifier so that relevance judgements compare directly against hits, and the corpus is embedded once and reused across configurations.

Three rules govern every number below. Legs that are compared are measured in one session on one machine with one warm model cache, because a cross-session ratio is an artefact. No claim about agent behaviour is made without a real language-model run; counts produced by a tokeniser alone are labelled as such. And every unmeasured cell is printed [? like this], so that a gap is visible in the document rather than only in its source.

6.2 Context Cost Against Corpus Size

The claim the paper rests on: what an agent must read before it can act does not grow with the corpus. Table 3 measures it over a synthetic tenant with 24 authored skills and instance counts spanning 10^2 to 10^5. Instances are bulk-inserted rather than written through the write path, since the quantity under test is a read.

Half of this is arithmetic and we should say so. The Tier-1 payload is byte-identical at every scale because the query is `WHERE page_type = 'skill'`: it cannot see instances. Reporting that as a discovery would be dressing a schema fact as a result. What it does buy is a guard — the harness asserts the equality, so if discovery ever starts touching instances, a test fails rather than a paragraph quietly becoming false.

Table 3: Tier-1 cost against corpus size. Tokens are `cl100k_base`; the harness records characters and the tokeniser is applied in the report. Latency is 30 samples per row, one session.

instances	10^2	10^3	10^4	10^5
Tier-1 tokens	744	744	744	744
Tier-1 p50 (ms)	0.74	0.74	0.76	0.81
Tier-1 p95 (ms)	0.88	0.91	0.80	1.16
whole corpus (tokens)	2.9k	29k	290k	2.9M
ratio avoided	3.9×	39×	390×	3898×

The part that was open is the latency, and it is nearly flat. Tier-1 shares the `pages` table with every instance, so the scan could have degraded. Across a thousand-fold increase in corpus it moves from 0.74 to 0.81 ms at p50, about 10%, with p95 noisier at 1.16 ms. Discovery stays sub-millisecond while the material it declines to read grows to 2.9 M tokens.

What this does not measure, and it is the important half. These are the KB's costs, not an agent's. Whether a language model actually exploits the cheap tier — reading descriptions, choosing a skill, expanding two pages instead of twenty — is a behavioural question, and the three-arm comparison against flat retrieval and whole-corpus-in-context that would answer it is specified in our plan and was not run. The predecessor's real-model run reached a correct answer in 1–2[P] tool calls at 12.4[P]× lower token cost than a flat baseline, on a 20-page prototype corpus. We are not restating that as a result for this system. What Table 3 establishes is that the mechanism is available at scale and costs a fixed 744 tokens to offer; what it cannot establish is that an agent takes it.

6.3 Retrieval Against Flat Retrieval Over Identical Bytes

Five configurations over the same corpus: flat retrieval with no types and no filter; single-pass typed retrieval; a two-pass variant with a truncated coarse dimension; a reranked variant; and both together. We report nDCG@10, recall@100, MRR@10, p50 and p95 latency and sustained throughput. Four of the five run today. The flat baseline is the new work, and it has to be a tuned one, so we sweep k and report the sweep beside the result rather than only the best cell. Result: [? Table 4].

6.4 Is It the Type, or Just a Filter?

A reviewer will ask whether the win is typing or merely metadata filtering, and we cannot answer it here. The ablation needs a corpus with more than one skill and queries that name a type; SciFact has neither, and escurel's own labelled corpus is the one Section 6's storage subsection reports as missing. Running a skill filter over a single-skill corpus compares a query against itself.

We can report the direction of one cost from the predecessor. It measured description-only indexing at 83.3%$^{\mathsf{p}}$ top-1 against a full-content baseline of 96.7%$^{\mathsf{p}}$. Disclosure buys tokens by giving up accuracy at the discovery step and recovering it in a second call. That trade is the mechanism rather than a side effect, and it is why Section 7 names corpora where it is a bad one.

6.5 The Falsifier We Could Not Run

The experiment that could refute the central claim is a plot of skill count against instance count over a real corpus's history. If skills grow with instances, discovery stops being cheap and the mechanism does not work.

We do not have the data. The corpora that would show it live on the deployed instance, and what is on hand is fixtures. We considered plotting those and decided against it: a curve drawn from a handful of example tenants would look like evidence for the one claim in this paper that most needs it, and would be worth less than the admission. The measurement stays specified and unrun, and Section 7 carries it as the open question it is.

6.6 The Projection Loop

Twenty events are captured into the inbox, the runner is started, and the clock runs until quiescence: every event `processed` on the gateway *and* the ledger no longer moving. Reading only the ledger would call it done while a write was still in flight. The harness is the echo adapter, so the numbers describe the loop rather than a language model's latency.

We were wrong twice about the bottleneck, and the arms are the record of it. The shipped per-tenant governor admits 120 runs per minute, which is exactly two per second, and the first arm measured 2.18 — a coincidence so clean that we took it for the explanation. Lifting the governor to 100,000 runs per minute moved throughput not at all (2.07). The second hypothesis was write contention, since all twenty events folded into one page behind a per-tenant write lock; giving each event its own target instance also moved nothing (2.03). What is left is an unmeasured residual of roughly 475 ms per event. We did not decompose it — the harness times quiescence, not poll wait against spawn against round trips — so the serial per-run cost we suspect is a hypothesis. The finding we can defend is narrower and still useful: neither mechanism we built to control throughput was controlling it here.

Table 4: Twenty events drained to quiescence, one session. The clock starts after capture, so this is drain time, not ingest-to-quiescence. The arms are not a clean isolation: lifting the governor also changes the failure profile.

target instances	governed 1	lifted 1	lifted 20
seconds to quiescence	9.17	9.66	9.87
events per second	2.18	2.07	2.03
ledger runs per event	1.25	1.75	1.75
stopped by a loop control	5	5	5
runs recorded failed	10	0	0
max folds of one event	1	1	1

Lifting the governor changed the failure profile, not the rate. Ten runs recorded `failed` under the default quota and none did with it lifted, while events per second stayed flat. The governor was not slowing the system down; it was converting contention into retries. Every arm still folded every event exactly once, which is the property that matters and the one the harness asserts.

Cascade control is visible in the run count. Twenty events produce 25 to 35 ledger runs — the follow-on `decision-record` events the cascade emits — of which five are dead-lettered by a loop control in every arm, and the fixture quiesces. We ran no control arm with the loop controls disabled, so this shows the cascade terminating here, not that the controls are what stopped it; E-9 is the test that exercises them directly.

6.7 Replay Under Kill

The sharper test. The runner is `SIGKILL`ed at a pseudo-random point in $[20, 100]$ ms — one event per trial, so this is a single-event crash probe and not a claim about multi-event contention — then restarted against the *same* durable ledger and given 60 seconds to converge.

One change to what we promised. The plan said final state byte-identical to an uninterrupted run. That comparison is not available: the echo harness keys its appended note on the event id, event ids are ULIDs, so two runs differ in their bytes for reasons that have nothing to do with replay. The invariant that carries the weight is effectively-once — the event appears in the instance body exactly once and ends `processed` — so that is what is measured.

The first version of this experiment measured nothing, and the protocol caught it. We wrote down in advance that fifty clean runs would be a warning rather than a result — that if nothing ever broke, the kills were probably

Table 5: Kill and replay, 100 trials, kill sampled from a deterministic sequence in $[20, 100]$ ms. Only the *in-flight* column tests replay; the rest is reported so the denominator is visible.

kill landed	trials	converged
before the run started	0	—
while in flight	**35**	**35**
after the run completed	65	65
duplicate folds		0
never processed		0

Table 6: The same entity on five substrates, one session, 50 samples per latency cell. Navigation is the fixed `resolve`/`expand`/ `neighbours` sequence. Search is each skill's *declared* capability, not a probe — see the text.

	markdown	sql	doc	rest	mcp
navigation steps ok	3/3	3/3	3/3	3/3	3/3
declared search	hybrid	late	hybrid	none	none
`expand` p50 (ms)	3.8	9.4	5.8	6.7	6.5
`expand` p95 (ms)	4.7	10.2	6.9	8.3	7.7
payload (bytes)	380	760	1572	436	427
writable via	page	—	—	instance	

not landing inside the write window. They were not. The first pass sampled kills from $[80, 1280]$ ms, all fifty trials came back clean, and the instrumentation showed why: *all fifty* kills landed after the run had already finished. A single event is folded in well under 80 ms, so the experiment had been measuring that killing an idle process does no harm. Narrowing the window to $[20, 100]$ ms put 35 of 100 kills genuinely in flight.

What we can and cannot say. Thirty-five in-flight kills converged, every one, with no duplicate fold and no event left unprocessed. That is evidence for E-8 at $n = 35$, not at $n = 100$, and 35 is a small number: it bounds the duplicate rate loosely, and a rare interleaving would not show up here. We report the denominator rather than the headline because the headline — "100/100 converged" — is true and would mislead.

6.8 Five Substrates, One Call Shape

The experiment that exercises the corollary along a fixed, known navigation path. One fixture tenant carries five skills describing the same entity, a customer, one per backend, against a Postgres in a container, a fixture PDF through `/ingest`, an axum CRM answering REST, and a JSON-RPC server answering MCP. The same sequence runs against each: `resolve` → `expand` → `neighbours`. Table 6 reports it.

The corollary holds for navigation from a known handle. All fifteen steps succeed: one set of verbs, in one order, routed to a local file, a Postgres query, a chunked PDF and two live services. Two limits on what that shows. The caller is not blind to the substrate — the skill segment of `[[customer_rest::acme]]` names it, which is the design, since it is the *type* that declares the backend — so what is identical is the call shape, not the caller's ignorance. And the harness addresses each instance by a handle it was given at creation, so this is navigation, not discovery of an arbitrary entity. Within those limits the harness asserts rather than records: a step that returns neither a body nor frontmatter fails the test.

The declared discoverability divides where the design says. This is a configuration read, not a behavioural result: the capabilities come back `hybrid` for markdown and documents, `late` for the SQL view, and `none` for both live proxies. We report the declaration rather than a similarity probe, and the reason is worth stating because it nearly produced a false result: the fixture runs a zero-vector embedder, under which every vector is identical and `search` returns essentially the whole corpus. A probe in that configuration reports that live backends are findable, which is the opposite of the truth. The number that was easy to collect was the wrong one.

The latency ordering is not the one we predicted. We expected the two materialised backends to answer from the local file in single digits and the live proxies to show a network round trip. Instead the *SQL view is the slowest cell* at 9.4 ms p50, half again the cost of either live proxy. The ordering is descriptive of one session — 50 warm calls in one process, no independent repeats, no variance reported, and the gaps are single-digit milliseconds — but the direction is the opposite of what we expected, and the mechanism is visible: every read crosses into Postgres through DuckDB's scanner while the proxies were answered by loopback services in the same process. Two things follow. The proxy figures are a floor: they exclude real network round-trip time, and a remote endpoint across the internet would move them by far more than the spread in this table. And "materialised" does not imply "local" — a SQL view is a live query wearing a materialised interface, which is a distinction the design documentation blurs and this measurement does not.

Failure behaviour is deliberately absent from this table: nothing here was measured with a source down. E-15 covers exactly one case — an unreachable `openapi` endpoint degrading rather than fabricating a body — and nothing in Table 6 extends to the others.

6.9 What the Storage Collapse Cost

We declared thresholds in advance before merging retrieval and relational state into one file: nDCG at or above 0.95 on a 460-block corpus and 0.90 on a 10,120-block stress corpus, p50 vector search within 15 ms, p95 filtered search within 50 ms at 100k instances. The Lance baselines were 0.975^{p}, 0.933^{p}, 4.3^{p} ms and 18.71^{p} ms.

We cannot report the comparison. The labelled corpus those thresholds were declared against is not in this repository, and it is not in the research tree either; it was never versioned with the code that depends on it. The gate has been open since May, and reopening it means rebuilding the corpus and its relevance judgements from scratch, which is a larger job

than the measurement it was meant to gate. We record it as a finding about process rather than about DuckDB: a pre-deployment gate whose fixture is not versioned alongside the code is a gate that will still be open when you deploy, and ours was.

6.10 Two Negative Results

Lexical retrieval collapsed where vector retrieval did not. On a 10,120-block stress corpus built by expanding 460 originals into synonym mutants, vector retrieval held at 0.933^{p} nDCG, four points below its small-corpus baseline. Lexical retrieval with a default tokeniser fell to 0.350^{p}, ranking mutants above originals on 17 of 30 queries. Part of this is an artefact of the stress construction, since real corpora do not contain twenty near-identical copies of every page, but we had assumed BM25 degrades gracefully under near-duplication, and it does not.

The default embedder our specification pins cannot be loaded by our own runtime. The configuration names EmbeddingGemma. The Candle transformer library has no Gemma-3 sentence-encoder path, so it will not load, and the actual default is a BERT model of the same dimensionality. Nothing failed loudly; a specification and an implementation simply disagreed for weeks while both were individually correct at the time they were written. We now treat "the specification names a model" and "the runtime can load that model" as separate claims requiring separate tests.

7 When You Should Not Use This

When the corpus has no natural types. If everything is one skill, discovery has nothing to enumerate, and you have paid a schema's authoring cost for a flat store's behaviour. The typing argument is a claim about domains with vocabulary, not about text in general.

When nobody will author skills. The type system is human-authored by construction. That is exactly why the skill count stops growing, and exactly why a team unwilling to write and curate skills gets the cost without the benefit.

When retrieval accuracy at the discovery step matters more than tokens. Description-only indexing gave up 13^{p} points of top-1 accuracy in exchange for the token budget. For a corpus small enough to read whole, that is a bad trade.

When writes outrun a single writer. One writer per tenant, serial within a tenant and parallel across them. This suits a knowledge base and does not suit an event firehose.

When you need sub-millisecond retrieval. An embedded HNSW index inside a shared analytical file is not a dedicated vector service, and we gave up measured latency for architectural simplicity on purpose.

When your sources are slow, or when you need to search them. A read that proxies a remote call inherits that call's tail latency, and live data is never indexed, so a remote instance is reachable by link and not by similarity. If either matters, materialise the source and accept the staleness you were trying to avoid — but note from Table 6 that materialising is not automatically the fast choice. A SQL view is a live query behind a materialised interface, and it was the slowest substrate we measured. Only the two backends that actually hold bytes locally, markdown and documents, are cheap by construction.

When you need cross-tenant federation, or hard multi-tenant isolation today. See E-18.

8 Related Work and Summary

Agent memory. MemGPT [10] pages context in and out of a window, which is a mechanism for managing an untyped store rather than a proposal for typing it. Mem0 [4] and Zep [13] extract entities and relations into a graph automatically; the contrast with our position is sharp and deliberate, since a machine-inferred schema is one nobody can read in the morning, review in a pull request, or ship as a pack. GraphRAG [6] summarises at index time and answers global questions well, but has no write loop: the index is built, not maintained. A-Mem [16] and the memory stream of generative agents [11] both organise episodic memory without giving it a type system a domain expert would recognise.

Skills, and what they left out. Agent Skills [2] introduced progressive disclosure for the procedural axis and stopped there. Our claim is that the same mechanism extends to episodic memory through one small generalisation, letting the instance declare its skill. In the domains we work in, that extension covers most of the volume, and volume is what sets the cost.

Federated access. Lakehouse catalogs [3, 5] and virtual query engines unify access to *data* without unifying the addressing of *knowledge*: they give one query language over many sources, where the corollary of Section 3.2 gives one referent space over many substrates, of which relational data is one. Event sourcing [7] is the direct ancestor of the second duality; what is new here is that the projection function is an agent, and its code is a document in the store it projects into.

Summary. A skill is a schema and a prompt; an instance is a row and a document; an event is a fact and a callback is its projection; and because the skill declares the backend, the substrate under an instance is a property of its type. That last point is the one we underestimated when we started, and the one we would keep if we had to discard the rest. Whether the token invariant survives contact with a corpus a thousand times larger than the one we have is the question we cannot answer yet, and Section 6 says precisely which measurement would settle it.

Availability. The specification, the eleven architecture decision records, the sixty-two discovered-problem notes cited throughout, and the plan this paper was written against are in the project repository.

References

[1] Anthropic. 2024. Model Context Protocol. https://modelcontextprotocol.io/.
[2] Anthropic. 2025. Equipping agents for the real world with Agent Skills. https://www.anthropic.com/engineering/equipping-

agents-for-the-real-world-with-agent-skills. Engineering blog post.
[3] Apache Software Foundation. 2024. Apache Iceberg: The Open Table Format for Analytic Datasets. https://iceberg.apache.org/.
[4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413* (2025).
[5] DuckDB Labs. 2025. DuckLake: SQL as a Lakehouse Format. https://ducklake.select/.
[6] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv:2404.16130* (2024).
[7] Martin Fowler. 2002. *Patterns of Enterprise Application Architecture*. Addison-Wesley. Event Sourcing pattern.
[8] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *TACL* 12 (2024), 157–173.
[9] Loro contributors. 2024. Loro: Reimagine State Management with CRDTs. *https://loro.dev/* (2024).
[10] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560* (2023).
[11] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proc. UIST*.
[12] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proc. SIGMOD*. 1981–1984.
[13] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956* (2025).
[14] Joachim Rosskopf. 2026. Progressive Disclosure of Agent Memory Through a Typed Skill–Instance Unification. Research draft, 2026-05-16.
[15] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS Datasets and Benchmarks.
[16] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. *arXiv:2502.12110* (2025).